

模拟退火算法 (Simulated Annealing) 的生动解读

你的名字

2025 年 8 月 30 日

1 核心思想：一个寻找最低谷的弹球

场景比喻：在广阔山脉中寻找最低点的弹球

想象一下，我们的任务是找到一片连绵起伏的山脉中海拔最低的那个山谷（全局最优解）。这片山脉地形复杂，有很多小山谷（局部最优解）。

模拟退火算法 (Simulated Annealing, SA) 的灵感来源于物理学中的金属退火过程。它巧妙地模拟了一个“弹球”寻找最低谷的过程，分为三个阶段：

- 高温阶段：**我们给弹球一个极高的初始温度，让它能量十足。
- 退火阶段：**我们让弹球遵循一定的规律，缓慢地降温。
- 低温阶段：**弹球的能量几乎耗尽，稳定在某个位置。

这个“先探索，后收敛”的策略，使得弹球有极大的概率最终停留在整个山脉最深的那个山谷，而不是随便一个小坑里。

2 算法的三个关键阶段

2.1 高温阶段：疯狂探索，跳出陷阱

在算法的初始阶段，温度 T 非常高。此时的弹球就像一个“超级赛亚球”，充满了无穷的能量。当把它扔到山脉的任意一个位置后，它会：

- 疯狂弹跳：**随机地、大幅度地向四周探索。
- 无视陷阱：**因为能量极高，即使它掉进了一个比较浅的小山谷（一个不够好的局部最优解），它也能毫不费力地一跃而出，继续去探索山脉的其他区域。

在算法中，这对应着一个关键机制：以很高的概率接受一个“更差”的解。这赋予了算法强大的全局搜索能力，避免了过早地陷入局部最优的“陷阱”中。

2.2 退火阶段：逐渐冷静，寻找方向

随着算法的进行，我们开始让温度 T 按照一个预设的速率（比如每次乘以 0.99）缓慢下降。这个过程就像给弹球“泼冷水”，让它逐渐冷静下来：

- **弹跳减弱**：弹球的能量在衰减，它的弹跳高度和范围都随之减小。
- **趋向更低**：它不再轻易地跳出深谷了。当它发现一个更低的位置时，会毫不犹豫地滚过去；但它仍然保留着一定的“活力”，偶尔也能跳出一个不太深的坑。

在算法中，这意味着随着温度 T 的降低，我们接受“更差解”的概率在动态地、非线性地减小。算法的搜索行为从“全局性的随意漫游”逐渐转变为“局部性的精细搜索”。

2.3 低温阶段：尘埃落定，锁定最优

当温度 T 降低到一个非常低的值（接近于 0）时，弹球的能量几乎完全耗尽。此时它已经变成了一个“佛系”的普通小球：

- **停止弹跳**：它失去了向上跳跃的能量。
- **滚入谷底**：它只会在当前所在的山谷中，顺着山势滚到最低点，然后静止下来。

在算法中，这意味着接受“更差解”的概率已经趋近于零。算法完全失去了随机探索的能力，只接受比当前解更好的解，从而在已找到的较优区域内进行最后的“收割”，锁定最终的解。

3 算法原理

3.1 物理背景

在热力学中，退火（Annealing）是指将固体材料加热至足够高的温度，然后缓慢冷却，使其内部粒子达到能量最低的稳定状态（晶格结构）的过程。在高温时，粒子具有较高的能量，可以自由运动并克服能量壁垒。随着温度的缓慢降低，粒子的运动逐渐减弱，并最终在能量最低的状态下达到平衡。

模拟退火算法正是借鉴了这一过程，将优化问题的求解过程与物理退火进行类比：

- **问题的解 (State)**：对应物理系统中的粒子状态。
- **目标函数 (Energy Function)**：对应粒子状态的内能 E 。我们的目标是最小化该函数。
- **控制参数 (Temperature)**：对应物理温度 T 。

3.2 Metropolis 接受准则

算法的核心在于其状态转移的概率机制，该机制遵循 Metropolis 准则。假设系统当前状态为 s_{curr} ，其能量（目标函数值）为 $E(s_{curr})$ 。算法通过某种邻域搜索机制生成一个新状态 s_{new} ，其能量为 $E(s_{new})$ 。

状态转移的决策如下：

1. 计算能量增量 $\Delta E = E(s_{new}) - E(s_{curr})$ 。
2. 如果 $\Delta E < 0$ ，意味着新状态优于当前状态，则该转移被无条件接受，即 $s_{curr} \leftarrow s_{new}$ 。
3. 如果 $\Delta E \geq 0$ ，意味着新状态是一个劣质解，该转移并不会被直接拒绝，而是以一个与温度相关的概率被接受。这个接受概率 P 定义为：

$$P(\Delta E, T) = \exp\left(-\frac{\Delta E}{T}\right)$$

这个概率性的接受准则是模拟退火算法能够跳出局部最优的关键。在初始高温 T 时，即使 ΔE 很大，接受概率 P 也可能接近 1，使得算法有充分的机会探索整个解空间。随着温度 T 的降低，接受劣质解的概率迅速衰减，使得算法最终收敛于一个高质量解。

4 算法流程

模拟退火算法的执行流程主要由三个部分组成：内循环、外循环和冷却策略。

- **内循环：**在某一固定温度 T 下，进行多次状态转移尝试，使得系统在该温度下达到“热平衡”。
- **外循环：**控制温度 T 按照预设的冷却策略（Cooling Schedule）缓慢下降。
- **冷却策略：**定义了初始温度 T_0 、终止温度 T_f 以及温度的衰减函数。最常用的衰减函数是指数衰减：

$$T_{k+1} = \alpha \cdot T_k$$

其中 α 是一个接近于 1 的常数（如 0.99），被称为冷却速率。

5 应用实例：求解 TSP 问题

在旅行商问题（TSP）中，我们需要找到一条访问 N 个城市并返回起点的最短哈密顿回路。

- **解空间：**所有可能的城市访问排列组合。

- 目标函数 $E(s)$: 给定一个排列 s (即一条路径), 其总距离。
- 邻域函数: 从当前路径 s_{curr} 生成新路径 s_{new} 的方法。常用的方法包括:
 - 随机交换 (Swap): 随机选择路径中的两个城市并交换其位置。
 - 2-Opt: 随机选择路径中的两条边, 断开并以唯一非交叉的方式重新连接, 等价于反转两条边之间的一段子路径。

以下 Python 代码实现了使用模拟退火算法求解 TSP 问题。

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import random
4 import math
5
6 def calculate_total_distance(route, coords):
7     """计算给定路线的总距离, 作为目标函数E(s)"""
8     total_dist = 0
9     for i in range(len(route)):
10         idx1 = route[i]
11         idx2 = route[(i + 1) % len(route)]
12         total_dist += np.linalg.norm(coords[idx1] - coords[idx2])
13     return total_dist
14
15 def simulated_annealing(coords):
16     """模拟退火算法主函数"""
17     num_cities = len(coords)
18
19     # 1. 初始化参数
20     T_initial = 1000.0
21     T_final = 1e-8
22     alpha = 0.99
23
24     # 2. 初始化解
25     current_route = list(range(num_cities))
26     random.shuffle(current_route)
27     current_distance = calculate_total_distance(current_route, coords)
28
29     best_route = current_route.copy()
30     best_distance = current_distance
31
32     T = T_initial
33
34     # 3. 算法主循环
35     while T > T_final:

```

```

36     # 在当前温度下迭代一次（简化内循环）
37
38     # 3.1 生成邻域解（采用随机交换）
39     new_route = current_route.copy()
40     i, j = random.sample(range(num_cities), 2)
41     new_route[i], new_route[j] = new_route[j], new_route[i]
42     new_distance = calculate_total_distance(new_route, coords)
43
44     # 3.2 Metropolis接受准则
45     delta_E = new_distance - current_distance
46     if delta_E < 0 or random.random() < math.exp(-delta_E / T):
47         current_route = new_route
48         current_distance = new_distance
49
50         if current_distance < best_distance:
51             best_route = current_route
52             best_distance = current_distance
53
54     # 3.3 降温
55     T *= alpha
56
57     return best_route, best_distance
58
59 # 主程序入口
60 if __name__ == "__main__":
61     cities = np.random.rand(20, 2) * 100
62     optimal_route, min_distance = simulated_annealing(cities)
63     print(f"Optimal route found: {optimal_route}")
64     print(f"Minimum distance: {min_distance:.4f}")

```

Listing 1: 模拟退火求解 TSP 问题的 Python 实现

6 总结

模拟退火算法是一种思想深刻、适用范围广的优化算法。它通过模拟物理退火过程，引入了随温度变化的“概率性跳出”机制，使其能够有效避免陷入局部最优解，尤其适合解决解空间复杂、崎岖的大规模组合优化问题。在数学建模中，当面对复杂的规划、调度或布局问题时，模拟退火是一个非常强大而可靠的工具。